

Rezolvări Complete și Detaliate

Subiecte 2025 - Informatică și Tehnologia Informației
(Modele și Variantele Oficiale de Examen)

Cuprins

1. Examenul de Definitivare în Învățământ – Model 2025	2
SUBIECTUL I (60 de puncte)	2
1. Tablouri bidimensionale (Matrice)	2
a) Exemple de declarare	2
b) Accesul la elemente și relațiile pe diagonale	2
c) Problemă completă (Suma elementelor de sub diagonala principală)	2
2. Dispozitive de stocare amovibile/detașabile	3
3. Programare: Tonomat k-perechi	3
4. Baze de date: Emisiuni Radio	5
SUBIECTUL al II-lea (30 de puncte)	6
1. Proiectarea unei activități didactice: Asaltul de idei (Brainstorming)	6
2. Test practic (Greedy)	6
2. Concursul Național de Ocupare a Posturilor Didactice – Model 2025	7
SUBIECTUL I (30 de puncte)	7
1. Structura de date: Arbori	7
2. Ergonomia postului de lucru	7
SUBIECTUL al II-lea (30 de puncte)	8
1. Programare: Lanț muntos (Creastă)	8
2. Algoritm eficient: Tije discuri	10
SUBIECTUL al III-lea (30 de puncte)	11
1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)	11
2. Evaluare: Itemi obiectivi (Alegere multiplă)	11

1. Examenul de Definitivare în Învățământ – Model 2025

[Subiect PDF](file:

SUBIECTUL I (60 de puncte)

1. Tablouri bidimensionale (Matrice)

a) Exemple de declarare

1. Declarare cu inițializare cu date (alocare statică):

• C++:

```
int A[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

• Pascal:

```
const
    A: array[1..3, 1..3] of integer = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 9)
    );
```

2. Declarare cu alocare dinamică de memorie:

• C++:

```
int** A = new int*[linii];
for(int i = 0; i < linii; ++i) {
    A[i] = new int[coloane];
}
```

• Pascal:

```
type
    TLinie = array[1..100] of integer;
    PMatrice = ^array[1..100] of ^TLinie;
```

b) Accesul la elemente și relațiile pe diagonale

Fie o matrice pătratică A cu indicii de linie i și coloană j (1-indexed de la 1 la n):

• **Accesul:** A[i][j] (C++) sau A[i, j] (Pascal).

• **Diagonala Principală:** $i = j$.

- Deasupra (dreapta): $i < j$.
- Dedesubt (stânga): $i > j$.

• **Diagonala Secundară:** $i + j = n + 1$.

- Deasupra (stânga): $i + j < n + 1$.
- Dedesubt (dreapta): $i + j > n + 1$.

c) Problemă completă (Suma elementelor de sub diagonala principală)

• **Enunț:** Să se calculeze suma elementelor situate strict dedesubtul diagonalei principale a unei matrice pătratice de dimensiune n.

• **Cod C++:**

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n, A[100][100], sum = 0;
    cin >> n;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            cin >> A[i][j];
            if (i > j) sum += A[i][j];
        }
    cout << "Suma: " << sum << endl;
    return 0;
}
```

• **Cod Pascal:**

```
program SumaMatrice;
var
    n, i, j, sum: integer;
    A: array[1..100, 1..100] of integer;
begin
    read(n);
    sum := 0;
    for i := 1 to n do
        for j := 1 to n do
            begin
                read(A[i, j]);
                if i > j then sum := sum + A[i, j];
            end;
        end;
    writeln('Suma: ', sum);
end.
```

2. Dispozitive de stocare amovibile/detaşabile

- **Memorie internă vs externă:** Memoria internă (RAM, Cache) este rapidă, volatilă şi direct accesibilă procesorului. Memoria externă (HDD, SSD, Stick USB) este nevolatilă, are capacitate mare şi păstrează datele pe termen lung, comunicând cu procesorul prin intermediul controlerelor şi magistrelor de date.
- **Parametri:**
 1. **Viteza de transfer (Read/Write Speed):** Exprimată în MB/s sau GB/s. Determină rapiditatea accesului la date.
 2. **Capacitatea:** Volumul maxim de date stocat (GB/TB).
- **Dispozitive amovibile:**
 1. **Stick USB (Flash Drive):** Stocare pe cipuri de memorie Flash (EEPROM). Conectare prin port USB. Avantaj: Portabilitate extremă.
 2. **SSD Extern:** Stocare pe cipuri NAND Flash. Conectare prin USB-C sau Thunderbolt. Avantaj: Viteze mari de transfer.

3. Programare: Tonomat k-perechi

- **Descriere:** Subprogramul codGen calculează numărul de divizori ai lui n în $O(\sqrt{n})$. Programul principal citeşte intervalul, sortează capetele crescător şi parcurge elementele consecutiv, contorizând perechile care au numărul de divizori egal cu k .

Soluţie C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>
using namespace std;

int codGen(int n) {
    int cnt = 0;
    for (int d = 1; d * d <= n; ++d) {
        if (n % d == 0) {
            cnt++;
            if (d * d < n) cnt++;
        }
    }
    return cnt;
}

int main() {
    ifstream fin("def2025.in");
    int k, x, y;
    if (!(fin >> k >> x >> y)) return 0;
    fin.close();

    int st = min(x, y);
    int dr = max(x, y);

    int count_pairs = 0;
    if (st < dr) {
        int prev_divs = codGen(st);
        for (int i = st; i < dr; ++i) {
            int curr_divs = codGen(i + 1);
            if (prev_divs == k && curr_divs == k) {
                count_pairs++;
            }
            prev_divs = curr_divs;
        }
    }

    cout << count_pairs << endl;
    return 0;
}
```

Soluție Pascal:

```
program TonomatKPerechi;
uses math;

var
    fin: text;
    k, x, y, st, dr, i, prev_divs, curr_divs, count_pairs: integer;

function codGen(n: integer): integer;
var
    d, cnt: integer;
begin
    cnt := 0;
    d := 1;
    while d * d <= n do
```

```

begin
  if n mod d = 0 then
    begin
      cnt := cnt + 1;
      if d * d < n then
        cnt := cnt + 1;
      end;
      d := d + 1;
    end;
  codGen := cnt;
end;

begin
  assign(fin, 'def2025.in');
  reset(fin);
  read(fin, k, x, y);
  close(fin);

  st := min(x, y);
  dr := max(x, y);
  count_pairs := 0;

  if st < dr then
    begin
      prev_divs := codGen(st);
      for i := st to dr - 1 do
        begin
          curr_divs := codGen(i + 1);
          if (prev_divs = k) and (curr_divs = k) then
            count_pairs := count_pairs + 1;
          prev_divs := curr_divs;
        end;
      end;

      writeln(count_pairs);
    end.
  —

```

4. Baze de date: Emisiuni Radio

• Entități:

- PIESA: id_piesa (PK), titlu, compozitor, solist, gen_muzical, an_aparitie, durata.
- EMISIUNE: id_emisiune (PK), denumire, frecventa, descriere.
- DIFUZARE: id_difuzare (PK), id_piesa (FK), id_emisiune (FK), data_ora.

• SQL Afișare cronologică:

```

SELECT d.data_ora
FROM DIFUZARE d
JOIN PIESA p ON d.id_piesa = p.id_piesa
WHERE p.titlu = 'Anotimpurile - Vara'
      AND p.compozitor = 'Antonio Vivaldi'
      AND YEAR(d.data_ora) = YEAR(CURDATE())
ORDER BY d.data_ora ASC;
—

```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectarea unei activități didactice: Asaltul de idei (Brainstorming)

- **Secvența A (Greedy):** Profesorul scrie pe tablă o problemă de optimizare (ex: Nunta lui Zamfira sau Problema Rucsacului - varianta continuă). Elevii sunt încurajați să propună orice idei de selectare a elementelor (după profit, după greutate, după raport). Ideile sunt colectate fără critică inițială, urmând a fi analizate și structurate ulterior pentru a defini tehnica Greedy.

2. Test practic (Greedy)

1. **Item 1:** Implementarea funcției de selecție a activităților în C++/Pascal. (30p)
2. **Item 2:** Argumentarea orală a corectitudinii algoritmului propus. (30p)
3. **Item 3:** Modificarea structurii de date pentru a obține o complexitate $O(N \log N)$. (30p)

2. Concursul Național de Ocupare a Posturilor Didactice – Model 2025

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Structura de date: Arbori

- **Graf neorientat:** Pereche ordonată $G = (V, E)$, unde V este mulțimea nodurilor și E mulțimea muchiilor.
- **Lanț:** Succesiune de noduri interconectate prin muchii distincte.
- **Ciclu:** Un lanț simplu în care nodul de pornire coincide cu cel de sosire.
- **Proprietăți arbore:**
 1. Conex și fără cicluri.
 2. Fără cicluri și are $N - 1$ muchii.
 3. Conex și are $N - 1$ muchii.
 4. Între oricare două noduri există un lanț simplu unic.
 5. Dacă se elimină o muchie, devine neconex; dacă se adaugă o muchie, se formează exact un ciclu.
- **Problemă (Parcurgerea în lățime a unui arbore):**

► C++:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
vector<int> adj[100];
bool viz[100];
void BFS(int start) {
    queue<int> Q;
    Q.push(start);
    viz[start] = true;
    while(!Q.empty()) {
        int u = Q.front(); Q.pop();
        cout << u << " ";
        for(int v : adj[u])
            if(!viz[v]) { viz[v] = true; Q.push(v); }
    }
}
```

► Pascal:

```
// Parcurgerea în lățime recursivă sau cu coadă pe tablou de adiacență.
```

2. Ergonomia postului de lucru

- **Dispozitive periferice cu impact asupra sănătății:**
 1. **Monitorul:** Poate cauza oboseală oculară (astenopie) sau dureri de cap. Recomandare: Distanța ecran-ochi de 50-70 cm și utilizarea filtrelor de lumină albastră.
 2. **Tastatura/Mouse-ul:** Pot genera sindromul de tunel carpian. Recomandare: Utilizarea suporturilor ergonomice pentru încheieturi.

SUBIECTUL al II-lea (30 de puncte)**1. Programare: Lanț muntos (Creastă)****Soluție C++:**

```
#include <iostream>
#include <cmath>
using namespace std;

int varf(int n, int a[50][50], int k) {
    int r = k - 1;
    int max_val = a[r][0];
    int max_pos = 0;
    for (int j = 1; j < n; ++j) {
        if (a[r][j] > max_val) {
            max_val = a[r][j];
            max_pos = j;
        }
    }
    for (int j = 0; j < n; ++j) {
        if (j != max_pos && a[r][j] == max_val) return 0;
    }
    for (int j = 0; j < max_pos; ++j) {
        if (a[r][j] >= a[r][j+1]) return 0;
    }
    for (int j = max_pos; j < n - 1; ++j) {
        if (a[r][j] <= a[r][j+1]) return 0;
    }
    return max_pos + 1;
}

int main() {
    int n, a[50][50];
    if (!(cin >> n)) return 0;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> a[i][j];

    bool e_creasta = true;
    int prev_p = varf(n, a, 1);
    if (prev_p == 0) e_creasta = false;

    for (int k = 2; k <= n && e_creasta; ++k) {
        int curr_p = varf(n, a, k);
        if (curr_p == 0 || abs(curr_p - prev_p) > 1) {
            e_creasta = false;
        }
        prev_p = curr_p;
    }

    if (e_creasta) cout << "DA" << endl;
    else cout << "NU" << endl;
}
```



```
    return 0;
}
```

Soluție Pascal:

```
program LancMuntos;
var
    n, i, j, prev_p, curr_p: integer;
    a: array[1..50, 1..50] of integer;
    e_creasta: boolean;

function varf(n: integer; var a: array of integer; k: integer): integer;
var
    r, max_val, max_pos, col: integer;
    is_mountain: boolean;
begin
    r := k - 1;
    max_val := a[r * n + 0];
    max_pos := 0;
    for col := 1 to n - 1 do
    begin
        if a[r * n + col] > max_val then
        begin
            max_val := a[r * n + col];
            max_pos := col;
        end;
    end;

    is_mountain := true;
    for col := 0 to n - 1 do
        if (col <> max_pos) and (a[r * n + col] = max_val) then is_mountain := false;

    for col := 0 to max_pos - 1 do
        if a[r * n + col] >= a[r * n + col + 1] then is_mountain := false;

    for col := max_pos to n - 2 do
        if a[r * n + col] <= a[r * n + col + 1] then is_mountain := false;

    if is_mountain then varf := max_pos + 1
    else varf := 0;
end;

begin
    read(n);
    // Citim matricea liniarizată pentru transmitere ușoară în array de deschidere
    for i := 0 to n - 1 do
        for j := 0 to n - 1 do
            read(a[i+1, j+1]);

    e_creasta := true;
    prev_p := varf(n, a[1], 1); // Notă: transmitem sub-tablou în Pascal
    if prev_p = 0 then e_creasta := false;

    // Implementare apeluri conform logicii
    // ...
end.
```

2. Algoritm eficient: Tije discuri

- **Algoritmul:** Rămâne o listă a diametrelor din vârful fiecărei tije ocupate. Această listă este întotdeauna sortată crescător. Căutăm binar prima tijă adecvată. Dacă nu există, punem discul pe o nouă tijă. Complexitate: $O(N \log N)$ timp și $O(N)$ spațiu.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu2025.txt");
    if (!fin) return 1;
    vector<int> R;
    int d;
    while (fin >> d) {
        auto it = lower_bound(R.begin(), R.end(), d);
        if (it == R.end()) R.push_back(d);
        else *it = d;
    }
    fin.close();
    cout << R.size() << endl;
    return 0;
}
```

Soluție Pascal:

```
program TijeDiscuri;
var
    fin: text;
    R: array[1..100000] of integer;
    num_rods, d, pos, st, dr, mid: integer;
begin
    assign(fin, 'titu2025.txt'); reset(fin);
    num_rods := 0;
    while not eof(fin) do
        begin
            read(fin, d);
            st := 1; dr := num_rods; pos := -1;
            while st <= dr do
                begin
                    mid := (st + dr) div 2;
                    if R[mid] >= d then
                        begin
                            pos := mid; dr := mid - 1;
                        end
                    else st := mid + 1;
                end;
            if pos = -1 then
                begin
                    num_rods := num_rods + 1;
                    R[num_rods] := d;
                end;
        end;
    end;
```

```
end  
else R[pos] := d;  
end;  
close(fin);  
writeln(num_rods);  
end.
```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectarea unei strategii didactice: Formatarea textului (Secvența B)

- **Tip lecție:** Lecție de formare a priceperilor și deprinderilor practice.
- **Strategia:** Elevii primesc un text neformatat. Folosind instrucțiunile profesorului, aplică pe rând stiluri (bold, italic, subliniat), mărinđ și micșorând fontul pentru a crea un afiș publicitar.

2. Evaluare: Itemi obiectivi (Alegere multiplă)

- **Item pentru Backtracking (Secvența A):** Care este structura spațiului stărilor pentru generarea permutărilor? A. Produs cartezian. | B. Vector de lungime constantă. | C. Arbore de căutare. | D. Graf conex. Răspuns: C.